

Visual Beverage Classification: An End-to-End Deep Learning Pipeline for Wine Variety and Beer Style Recognition

Deep Learning on Computational Accelerators - Final Project

Contents

1	Introduction	2
1.1	The Human Perceptual Gap	2
2	YOLO Detection and Cropping	3
2.1	Motivation	3
2.2	Detection	3
2.3	Cropping	3
2.4	Verification	3
2.5	Detection Result	4
3	Dataset Preparation	4
3.1	Final Dataset	4
3.2	Class Imbalance and Weighting	5
3.3	Data Augmentation	5
3.4	Implementation: custom code to GPU native augmentation.	6
4	Model, Training, and Evaluation	6
4.1	Dataset Splitting	6
4.2	Model Architecture	6
4.3	Loss, Optimizer, and Training Setup	7
5	Results: What the Numbers Taught Us	8
5.1	Some of the Experiments	9
5.2	Where We Ended Up: Final Results Across All Three Problems	10
5.3	Comparison Against Pretrained Models	12
6	Limitations	15
7	Conclusions	15

1 Introduction

The goal of this project is to build an end-to-end deep learning pipeline capable of classifying beverages directly from the appearance of the liquid in the glass, distinguishing between five beer types (*Cider*, *Wheat*, *Lager*, *IPA*, *Stout*) and seven wine varieties (*Orange Wine*, *Rose*, *Sauvignon Blanc*, *Chardonnay*, *Cabernet Sauvignon*, *Pinot Noir*, and *Malbec*).

To build this classifier, we collected and curated our own dataset. Wine images were gathered in two ways: by personally photographing glasses of wine whenever the opportunity arose, and by sourcing images from Pinterest, Instagram, Google, and dedicated wine websites, specifically searching for photographs showing a glass of wine alongside its bottle. This pairing allowed us to verify the wine’s variety from the bottle label, providing reliable ground truth labels for each image. The beer dataset was acquired from Untappd, a dedicated platform for beer discovery and logging, which provided labeled images across a range of beer styles.

1.1 The Human Perceptual Gap

Identifying a wine variety or beer style purely by sight is genuinely hard, not just hard for us, but hard for experts. A Chardonnay and a Sauvignon Blanc, poured side by side, can look essentially identical. Many lager and cider styles share the same clear golden colour. Stout is the only one that gives itself away immediately.



Chardonnay



Sauvignon Blanc

Figure 1: Chardonnay vs. Sauvignon Blanc .

In a well known 2001 study, Morrot et al. presented 54 trained wine tasters with a white wine that had been dyed red using an odorless colorant. Almost without exception, the tasters described it using red wine vocabulary. The color had completely overridden their sense of smell. The visual signal was so dominant it rewrote their sensory experience of the drink entirely.

The same dynamic shows up in beer. Reinoso Carvalho et al. (2017) took a single pale beer and served it in two versions, one unchanged, one artificially darkened to look like a dark ale. Both tasted identical in blind conditions, but the darkened version was consistently rated as heavier, more bitter, and more expensive, before participants had even tasted it. The colour of the liquid in the glass told people what they were about to experience more than the actual taste did.

What both studies point to is a consistent gap: human perception of beverages is unreliable when visual cues are subtle or misleading, even for people who should know better. A neural network trained on enough examples might learn the fine grained hue, clarity, and light transmission patterns that distinguish one class from another, features that are consistent within a class but too subtle for the human judgment.

This perceptual gap motivates a data driven approach: a trained neural network can learn fine grained visual features, hue gradients, clarity, and light transmission patterns, that are consistent within a class yet subtle enough to lie beyond reliable human recognition.

2 YOLO Detection and Cropping

2.1 Motivation

The photographs we collected were rarely clean. Feeding such images directly into a classifier would risk the network learning unintended features, such as the bottle label rather than the beverage itself, making the training less reliable.

To address this, we designed an automated detection and cropping stage: instead of manually framing every photograph, we applied a pre trained object detector to locate the glass/mug in each image and extract only that region.

2.2 Detection

We used YOLO11 nano (`yolo11n.pt`), The model was applied to each image with a confidence threshold of 0.25, meaning only detections the model was at least 25% confident about were retained. Detections were filtered to glass related COCO categories: *cup*, *wine glass*, *glass*, *mug*, *beer*, *cup of beer*, *beer mug*, and *glass of wine*.

For each image, the detector produced a list of detections, each carrying:

- class label
- confidence score indicating detection certainty
- bounding box defined by pixel coordinates ($x_{\min}$, $y_{\min}$, $x_{\max}$, $y_{\max}$)

All results were serialized to a timestamped JSON file, decoupling the detection step from the cropping step and allowing the two to be run independently.

2.3 Cropping

With the detection JSON in hand, a second script read each bounding box and extracted the corresponding rectangular region from the original image. Bounding box coordinates were validated and clipped to the image boundaries before cropping, preventing errors from imprecise detections near image edges. Each crop was saved into a class specific subfolder, automatically organizing the output by detected category.

2.4 Verification

To visually inspect the quality of detections before committing to the cropped dataset, we used a dedicated visualization script that drew colored bounding boxes with class labels

and confidence scores directly onto copies of the original images, leaving the originals untouched. This allowed us to quickly scan for missed detections, false positives, or poorly framed boxes, and adjust the confidence threshold accordingly.

2.5 Detection Result



Figure 2: YOLO detection and cropping applied to an orange wine image. The original photograph (left) contains the bottle and background clutter.

3 Dataset Preparation

3.1 Final Dataset

After the collection, labeling, and cropping stages, the full dataset comprises 4,451 images across 12 classes: 7 wine types and 5 beer types. The complete dataset is publicly available at this [Google Drive folder](#).

Category	Class	Image Count
Wine	Orange Wine	211
	Rose	589
	Sauvignon Blanc	197
	Chardonnay	206
	Cabernet Sauvignon	188
	Pinot Noir	147
	Malbec	155
Beer	Cider	579
	Wheat	619
	Lager	382
	IPA	494
	Stout	684
Total		4,451

Table 1: Image counts per class after collection, labeling, and cropping.

3.2 Class Imbalance and Weighting

The dataset contains two sources of class imbalance. Within wine, Rose contains approximately 600 images, roughly three times the size of other wine varieties (~ 200 each), because it was sourced from a publicly available dataset. Within beer, Stout and Wheat each exceed 600 images while Lager sits at 382. If left unaddressed, this would implicitly reward the model for predicting the over represented classes more often, inflating overall accuracy while masking poor recall on minority classes. To counteract this, we applied class weights inversely proportional to frequency, so that a misclassification of a rare class incurred a proportionally larger penalty.

3.3 Data Augmentation

To further increase the effective diversity of the training set and reduce overfitting, we applied a suite of augmentations to the training images. Each augmentation was chosen to simulate realistic variation present in our collection, different cameras, lighting conditions, and shooting angles, without distorting the visual features that actually distinguish one beverage from another.

Augmentation	Rationale
Horizontal Flip	A glass photographed from the left is visually identical to one photographed from the right.
Rotation	Simulates handheld photography where the camera is not perfectly level. We use a small angle range which keeps glasses upright and realistic.
Perspective Warp	Simulates overhead shots, angled bar photographs, and varying shooting distances.
Color Jitter (brightness, contrast, saturation)	Accounts for the wide variation in lighting across collection sources: restaurant lighting, daylight, indoor bars, and web images all produce different exposures and color casts.
Gamma Correction	Simulates camera exposure differences. Unlike brightness scaling, gamma affects highlights and shadows non linearly, better matching real camera variation.
Gaussian Blur	Phone photos are often slightly out of focus, especially close up shots of foam or the glass surface. Training with blurred samples prevents over reliance on fine textures that may be absent at inference time.

Table 2: Augmentations applied during training and their rationale.

Discarded augmentations. During early experimentation we also applied random hue shifts and grayscale conversion to the training images. Both were removed for the same reason: hue is the *primary* discriminative feature between classes, the amber of an IPA, the deep red of a Cabernet, the pale gold of a Chardonnay. Randomly shifting hue, or stripping color entirely by converting to grayscale, taught the network to ignore the

very signal it should be learning, degrading classification performance. All other color augmentations were kept because they affect lighting and exposure, not the underlying hue of the beverage.

Also, ColorJitter and gamma correction were later removed, which will be discussed in Section 5.1.

3.4 Implementation: custom code to GPU native augmentation.

We went through two earlier implementations before settling on the final one. The first wrote all augmentations by hand in NumPy. The motivation was simplicity, we had full visibility into every operation and could verify each transform precisely. In practice, though, it was too slow: NumPy runs on the CPU and processes images sequentially, so augmenting the images before each training epoch became a bottleneck.

The second attempt switched to PIL’s built in image operations, which are backed by compiled C code and noticeably faster per operation. We chose this route hoping to close the gap without a large refactor. It helped, but the problem still remained: augmentation still ran on the CPU, one image at a time, so training throughput was still limited by preprocessing rather than the GPU.

We then switched to Kornia, a computer vision library built directly on PyTorch, which runs augmentations as batched tensor operations on the GPU. The full pipeline applies to an entire batch at once, on the same device as the model, with no CPU-GPU transfers in the critical path.

4 Model, Training, and Evaluation

4.1 Dataset Splitting

We split the data into three non overlapping sets: 70% for training, 15% for validation, and 15% for the final test. A CNN trained from scratch needs as many labeled examples as it can get, so we weighted the split heavily toward training. We were careful to split per class rather than over the whole dataset at once, ensuring every split preserved the original class distribution.

4.2 Model Architecture

We built our convolutional neural network from scratch: a stack of convolutional layers that progressively extract visual features, followed by a small fully connected head that maps those features to a class prediction.

Concretely, the network runs six convolutional layers with channel widths of 32, 32, 64, 64, 128, and 128. After every two convolutional layers we insert a max pooling step, which halves the spatial resolution and forces the network to build increasingly abstract representations. Every convolutional layer is followed by batch normalization and a ReLU activation. Batch normalization keeps activations in a well behaved range throughout training, which makes the whole process more stable and tends to speed up convergence.

After the convolutional layers, the feature maps are flattened into a single vector and passed through two fully connected layers of size 512 and 256 respectively. Each of those layers has dropout with a probability of 0.5, which is the main mechanism keeping the model from memorizing the training set. The final layer produces one output per class.

The whole architecture is driven by `config.json`: channel counts, pooling frequency, hidden layer sizes, activation function, dropout rate. This allowed us to experiment with different configurations without touching any of the model code.

Stage	Description
Input	$3 \times 224 \times 224$ RGB image
Conv Block 1	Conv2d(3, 32) $\rightarrow$ BatchNorm $\rightarrow$ ReLU
Conv Block 2	Conv2d(32, 32) $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ MaxPool(2×2)
Conv Block 3	Conv2d(32, 64) $\rightarrow$ BatchNorm $\rightarrow$ ReLU
Conv Block 4	Conv2d(64, 64) $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ MaxPool(2×2)
Conv Block 5	Conv2d(64, 128) $\rightarrow$ BatchNorm $\rightarrow$ ReLU
Conv Block 6	Conv2d(128, 128) $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ MaxPool(2×2)
Flatten	Collapse spatial dimensions to a feature vector
FC Layer 1	Linear(512) $\rightarrow$ ReLU $\rightarrow$ Dropout(0.5)
FC Layer 2	Linear(256) $\rightarrow$ ReLU $\rightarrow$ Dropout(0.5)
Output	Linear(12) - one logit per class

Table 3: Layer by layer architecture of the custom CNN.

4.3 Loss, Optimizer, and Training Setup

Loss function: We used cross entropy loss, but with two adjustments on top of it. The first is the class weighting we described earlier, The second is label smoothing with a value of 0.1, which softens the hard targets slightly. This discourages the model from becoming overly confident in its predictions, which tends to help when several classes look very similar to each other.

Optimizer: We started with plain Adam, which worked fine, but switched to AdamW once we started tuning regularization more seriously. The difference is in how weight decay is handled. In standard Adam, L2 regularization interacts with the adaptive learning rates in a way that makes it weaker than it should be. AdamW decouples the two so weight decay actually behaves as intended. We used a learning rate of 2×10^{-4} and a weight decay of 5×10^{-4} . The learning rate is fairly conservative, which we found worked better for this problem, the features that distinguish one wine from another are subtle enough that large gradient steps tended to overshoot them.

Early stopping: We monitored the validation loss throughout and stopped when it had not improved for 75 consecutive epochs. At that point we restored the weights from the best epoch seen so far. This meant we were never committing to a model that had started to overfit.

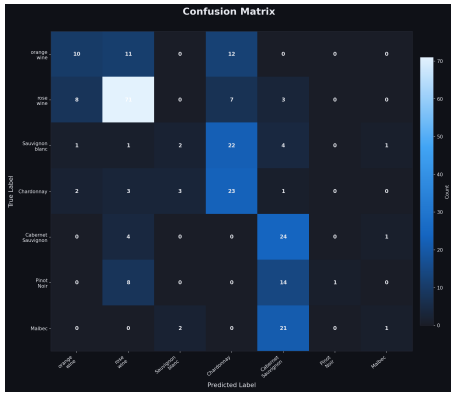
Cross validation: Beyond the standard train/val/test setup, we also ran 5 fold cross validation. We merged the training and validation sets into a single pool, divided it into five stratified folds, and trained the model five separate times, each time using four folds for training and the remaining fold to check generalization. The final model was then trained on the full pool and evaluated on the test set.

5 Results: What the Numbers Taught Us

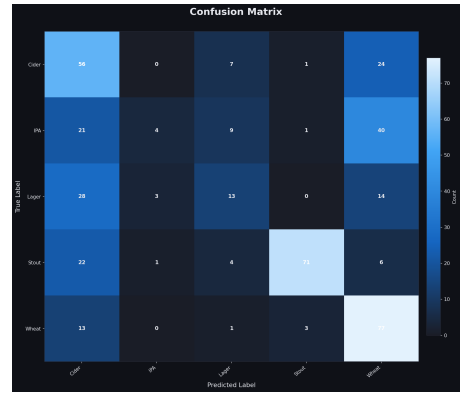
Running forty five experiments produced far more than a final accuracy number. The numbers alone do not tell the full story, each experiment taught us something specific about the problem, and those shaped every decision that followed. This section walks through the most important ones.

The first thing we did after running the pipeline was look at the confusion matrices, not the accuracy number.

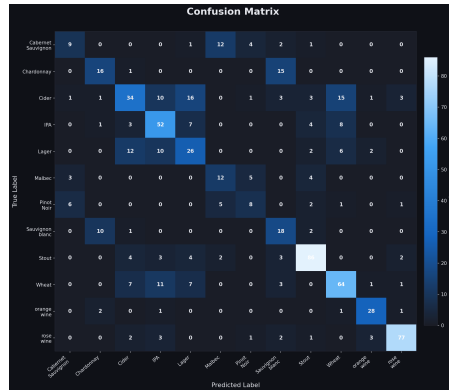
We ran baselines on all three formulations at once: wine alone, beer alone, and all 12 classes combined. Our first wine model reached 50.6%, first beer was 52.7%, and the combined 63.2%. All three were bad, but in different ways.



Wine - (50.6%)



Beer - (52.7%)



Combined - (63.2%)

Figure 3: Confusion matrices for the three initial baselines.

Wine The white wines could not be separated at all, Sauvignon Blanc was predicted as Chardonnay on 24 out of 31 test images ($F1 = 0.11$). The reds were a different kind of mess, not collapsing into one attractor but scattering, with Malbec at $F1 = 0.07$ and Pinot Noir at 0.08. The model had found a rough pale/red/pink partition but had zero ability to discriminate within any of those families.

Beer Stout has no visual neighbour but the pale styles had completely collapsed: IPA sent 40 of 75 images to Wheat and 21 to Cider, finishing at $F1 = 0.10$. Lager sat at 0.28. IPA, Lager, and Cider all share a clear golden appearance and the model could not separate them at all.

Combined The most striking observation is what did not break: the model did not confuse wine for beer in any systematic way. The matrix looks like two separate problems

put together, the white wine collapse and red wine scatter from the first wine version, and the pale style collapse from the first beer version, both reappearing intact. The only new failure was occasional Sauvignon Blanc/Cider confusion.

5.1 Some of the Experiments

Turning off ColorJitter and Gamma correction (V7 $\rightarrow$ V8):

V7 had the full augmentation suite active, including ColorJitter perturbing brightness, contrast, and saturation, and gamma correction. Test accuracy was 44.1%, the same collapse as before: reds scattered, whites pulled toward Chardonnay, and only Rose wasn't affected.

We turned off ColorJitter and gamma in more of a process of elimination step than a confident hypothesis. V8 came back at 64.8%. A jump of more than 20 percentage points from disabling two augmentations.

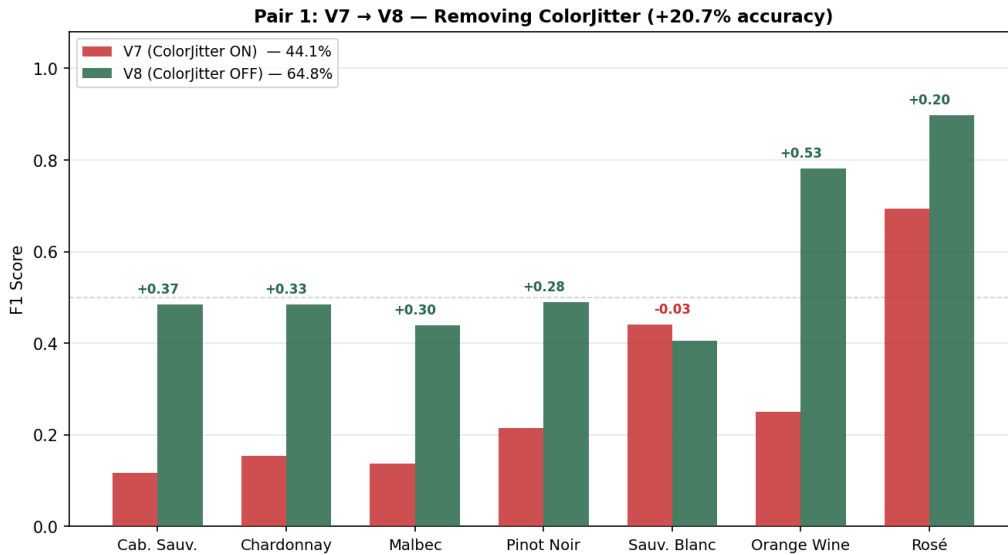


Figure 4: Per class F1 scores for V7 (ColorJitter enabled, 44.1%) vs. V8 (ColorJitter disabled, 64.8%)

Once we saw the result, the explanation was obvious in retrospect. ColorJitter randomly shifts hue and saturation at every training step, fine for most vision tasks, but for wine, hue is the class. A Malbec is near black. A Cabernet is deep red. By scrambling those signals during training, we had been teaching the model to ignore the very feature it needed to learn.

Input resolution (V10 $\rightarrow$ V11):

We tested whether giving the model more pixels would help resolve the subtle differences between Chardonnay and Sauvignon Blanc. V10 used 384×384 input, V11 dropped back to 224×224 . Overall accuracy was almost identical, 63.98% for both. But V10 had completely failed on Chardonnay: $F1 = 0.00$, While at the lower resolution Chardonnay partially recovered to $F1 = 0.28$.

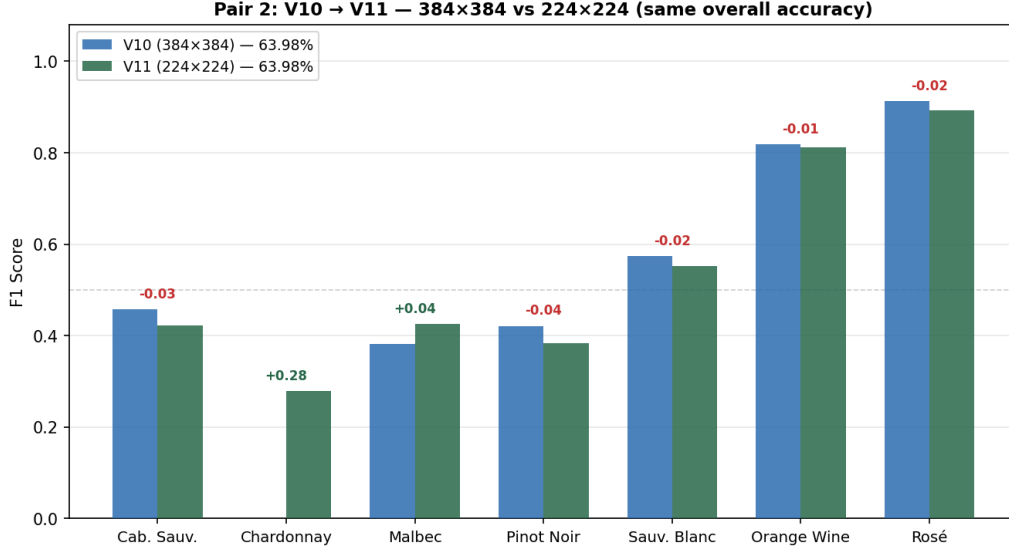


Figure 5: Per class F1 for V10 (384×384) vs. V11 (224×224)

Our hypothesis was that many of our images were collected at modest resolutions and upscaled to 384×384 , so the extra pixels are interpolation artefacts rather than real visual content. The model overfitted to that synthetic texture with more precision than it needed.

5.2 Where We Ended Up: Final Results Across All Three Problems

After iterating through configurations, we arrived at final models for all three formulations. The curves below show the best wine and beer models across all five cross validation folds.

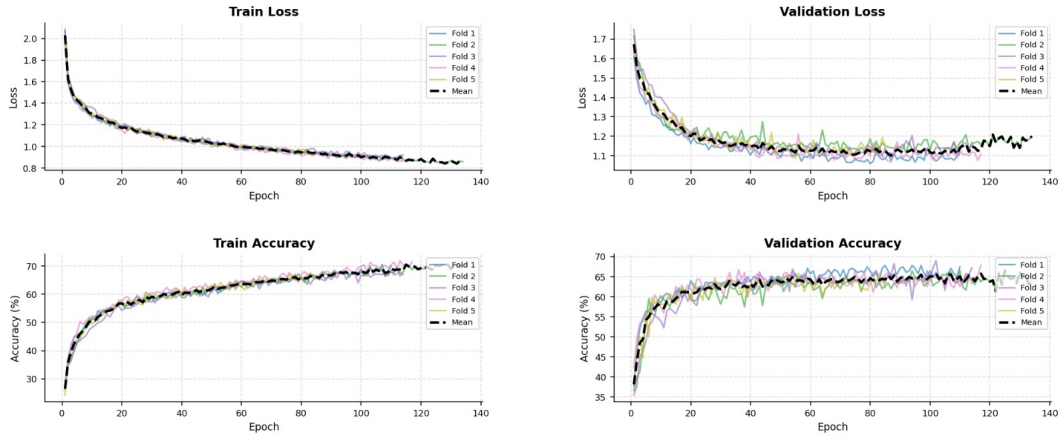


Figure 6: Training and validation loss curves for the best wine model across all five cross validation folds.

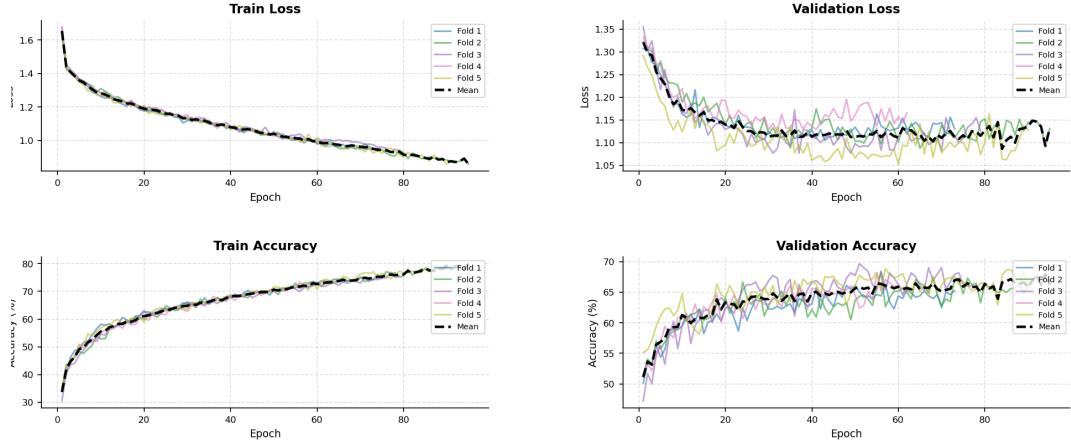
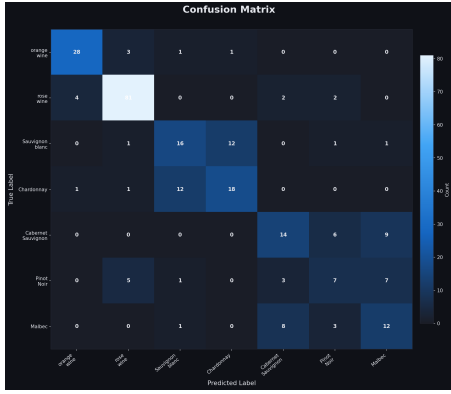
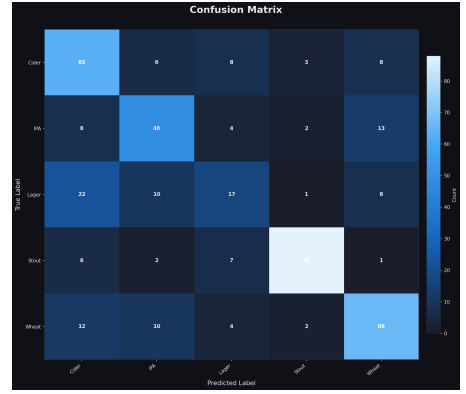


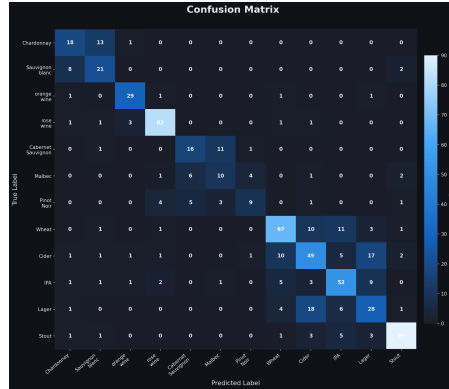
Figure 7: Training and validation loss curves for the best beer model across all five cross validation folds.



Wine - (67.4%)



Beer - (67.3%)



Combined - (69.3%)

Figure 8: Final confusion matrices across all three problem formulations.

Wine (67.4%).

Both families of confusion had been substantially resolved. Sauvignon Blanc went from $F1 = 0.00$ to 0.52, Malbec from 0.08 to 0.45, Pinot Noir from 0.17 to 0.33. Orange Wine and Rose were confidently classified at $F1 = 0.85$ and 0.90.

Beer (67.3%).

The pale style cluster had partially separated. IPA climbed from $F1 = 0.10$ to 0.64, Cider reached 0.63, Wheat 0.69. Lager remained the hardest class at 0.35, still partially

absorbed by Cider but no longer invisible. Stout stayed strong at 0.88.

Combined (69.3%).

The final 5 fold cross validated model improved on both the first combined attempt (63.2%) and on the best separate models. The more interesting result is not the headline number but what the matrix shows: pulling all 12 classes into a single classifier did not force a trade off between the two domains. The wine confusions that were resolved in the wine only experiments stayed resolved, and the beer pale style cluster that had partially separated stayed separated.

Reading the three matrices together.

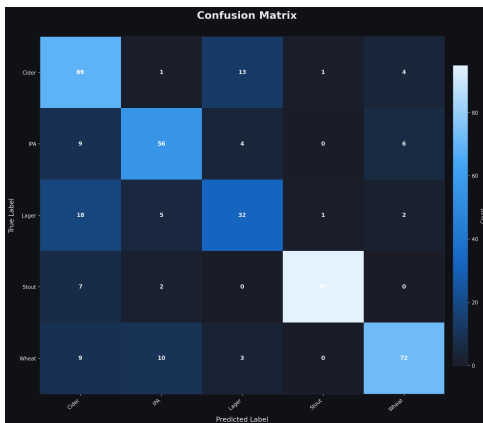
Wine improved within its family confusions, and those gains transferred cleanly into the 12 class combined model. Beer’s pale style cluster, partially separated in both the beer only and combined models. Lager and Pinot Noir are the remaining hard cases in their respective domains. In our opinion, that is a data problem, not an architecture problem. Given more labeled images for these classes, the diagonal entries would fill in.

5.3 Comparison Against Pretrained Models

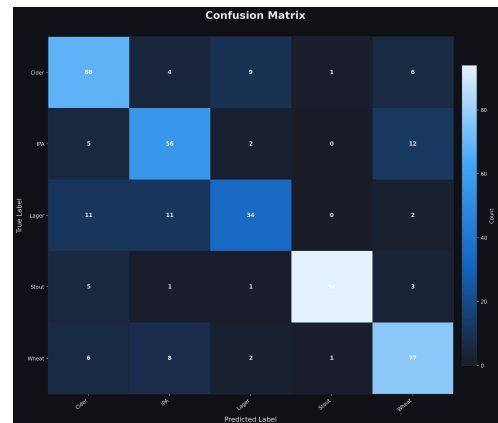
One final question: how does our custom CNN compare to models that started with the benefit of pretraining on millions of images?

To put our results in context, we fine tuned two pretrained models across all three problem formulations: beer alone, wine alone, and the full 12 class combined task. The two models were ResNet-50 and EfficientNet-B7, both pretrained on ImageNet. We will now show the results

Beer:



ResNet-50 beer (77.3%)



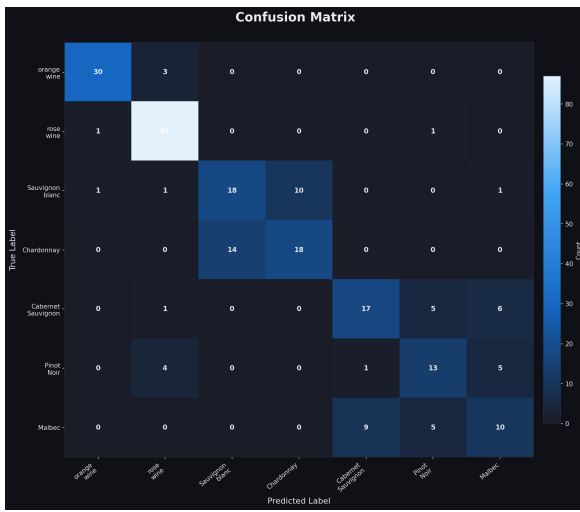
EfficientNet-B7 beer (78.5%)

Figure 9: Pretrained baselines fine tuned on the beer dataset

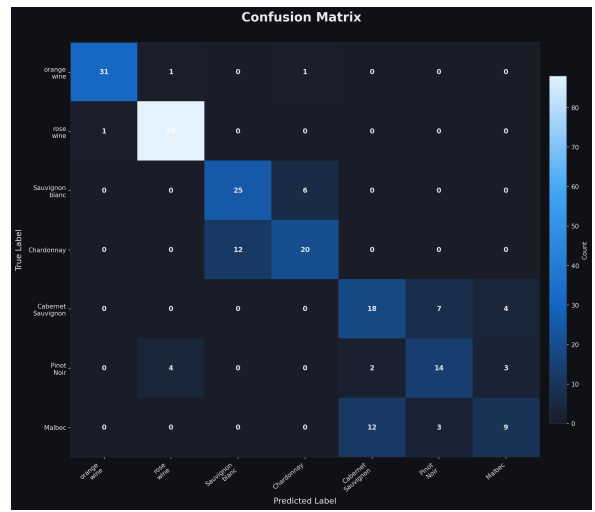
Table 4: Per class F1 on the beer dataset: our custom CNN vs. pretrained baselines.

Class	CNN	ResNet-50	EfficientNet-B7
Cider	0.63	0.69	0.74
IPA	0.64	0.75	0.72
Lager	0.35	0.58	0.64
Stout	0.88	0.95	0.94
Wheat	0.69	0.81	0.79
Test Accuracy	67.3%	77.3%	78.5%

Wine:



EfficientNet-B7 wine (73.9%)



ResNet-50 wine (78.5%)

Figure 10: Pretrained baselines fine tuned on the wine dataset

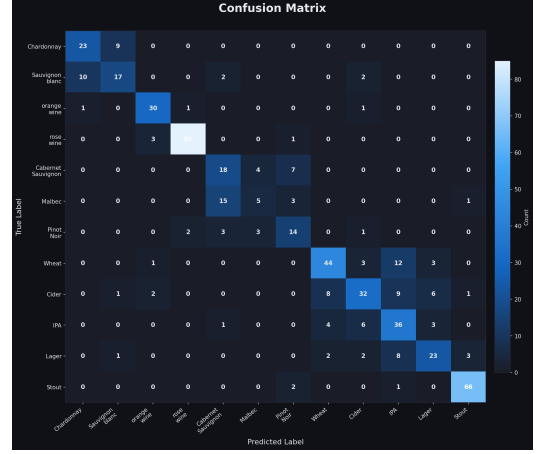
Table 5: Per class F1 on the wine dataset: our custom CNN vs. pretrained baselines.

Class	CNN	EfficientNet-B7	ResNet-50
Cabernet Sauvignon	0.50	0.61	0.59
Chardonnay	0.57	0.60	0.68
Malbec	0.45	0.43	0.45
Pinot Noir	0.33	0.55	0.60
Sauvignon Blanc	0.52	0.57	0.74
Orange Wine	0.85	0.92	0.95
Rose	0.90	0.94	0.97
Test Accuracy	67.4%	73.9%	78.5%

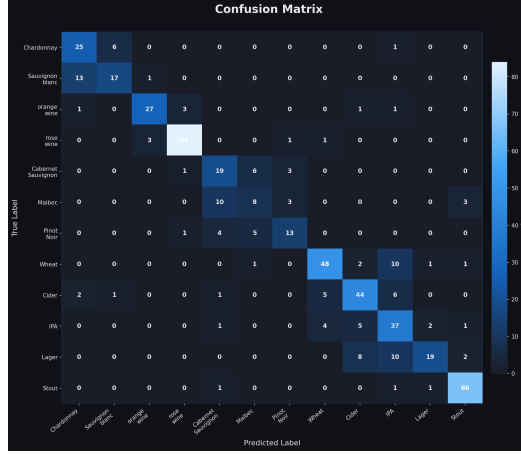
Combined:



CNN combined (69.3%)



ResNet-50 combined (72.6%)



EfficientNet-B7 combined (75.2%)

Figure 11: Our custom CNN vs. pretrained models on the full 12 class combined problem.

Table 6: Per class F1 on the combined 12 class dataset: our custom CNN vs. pretrained baselines.

Class	CNN	ResNet-50	EfficientNet-B7
Cabernet Sauvignon	0.57	0.53	0.58
Chardonnay	0.56	0.70	0.68
Cider	0.56	0.60	0.74
IPA	0.68	0.62	0.64
Lager	0.47	0.62	0.61
Malbec	0.41	0.28	0.36
Pinot Noir	0.47	0.56	0.60
Sauvignon Blanc	0.59	0.58	0.62
Stout	0.89	0.94	0.93
Wheat	0.73	0.73	0.79
Orange Wine	0.85	0.87	0.84
Rose	0.91	0.96	0.94
Test Accuracy	69.3%	72.6%	75.2%

What the gap tells us:

The pattern is consistent across all three formulations. Stout, Rose, and Orange Wine sit near the ceiling for everyone, the visual signal is strong enough that any model finds them quickly. The pretrained advantage concentrates in exactly the hard cases: Lager, Pinot Noir, and Sauvignon Blanc, the classes where our dataset is smallest and the differences are the most subtle.

The combined table tells us that our CNN outperforms both pretrained models on IPA (0.68 vs. 0.62 and 0.64) and matches ResNet-50 on Wheat (0.73 each). On Malbec, our CNN (0.41) actually beats both ResNet-50 (0.28) and EfficientNet-B7 (0.36). The pretrained models have their advantage on Chardonnay, Lager, and Pinot Noir, which are precisely the classes with fewest training images and the most subtle visual distinctions.

The gap between our CNN and The other pretrained models exists but that is the expected cost of starting from zero with limited data, and it tells us that the pipeline we built, the augmentation decisions, the class weighting, the careful tuning, got our small hand collected dataset to perform at the same level as heavily pretrained alternatives.

6 Limitations

The most consistent limitation was dataset size. The beer dataset averages around 600 images per class, while most wine varieties sit at 150-210. This asymmetry meant the wine models were working with roughly a third of the data the beer models had, which likely accounts for much of the performance gap between the two domains. Within wine, the hardest classes: Pinot Noir, Malbec, and Sauvignon Blanc are also the smallest, and the per class F1 scores track this directly.

Data collection bias is a subtler problem. Wine images were sourced from Pinterest, Instagram, and dedicated wine websites, photographs that tend to be well lit and styled. Beer images came uniformly from Untappd, which has its own photographic aesthetic. A model trained on this distribution may struggle with images that look different: bad lighting, partial fills, or glasses photographed at a bar rather than staged for a post. We have no way to measure this gap without an independent set collected under different conditions, which we did not have.

7 Conclusions

This project was built from the ground up. We collected the images ourselves, verified them ourselves, and trained a classifier from scratch with no pretrained weights.

Given that, the results surprised even us. Models like EfficientNet-B7 and ResNet-50 come loaded with weights learned from tens of millions of images, Whereas we had only 4,451. Yet the gap between our CNN and the pretrained baselines is roughly 6 points on the combined dataset.

What we learned goes beyond any single experiment. Our intuition about the problem guided decisions that the numbers later confirmed. Domain knowledge and machine learning pulled in the same direction, and that combination moved the model further than either would have alone.

We also learned to diagnose before combining. Training wine and beer separately first, understanding how each domain was failing, and only then bringing them together was not the obvious path but it was the right one.